Module 5: Analysis

Semantic Analysis

Imagine, like, the Evaluator, but like, without evaluating.

0.

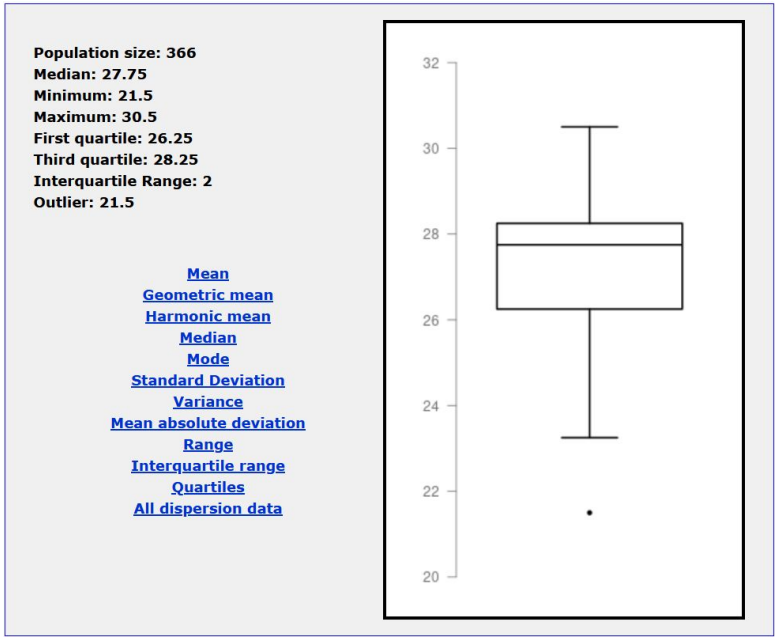
Assignment Review

Late but not forgotten!

Exercise 2: Error Handling

Average 91.3%, Median 92.5%

-
- Overcomplication from manually handling Exceptions/Results.
- Common issues:
  - Directly accessing our mock DB instead of using UrlModel.
  - Handling invalid body fields.
  - Handling conflicting aliases.

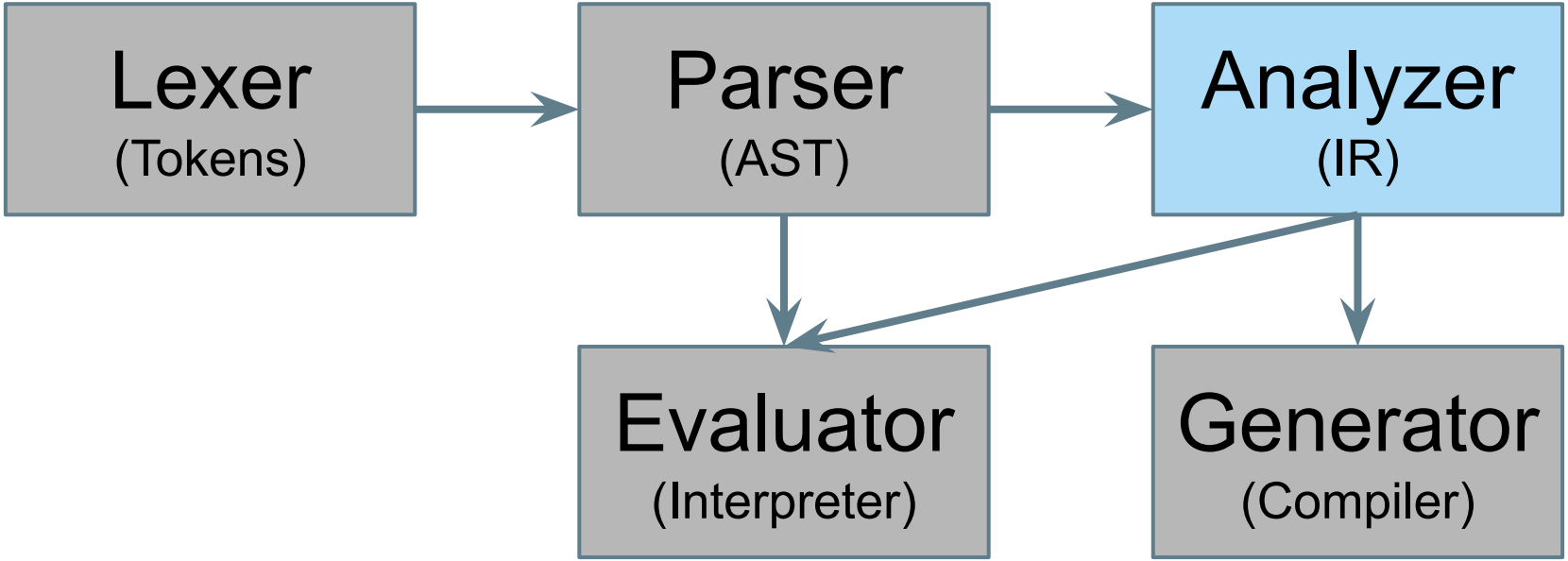


1.

Semantic Analysis

I don’t want to create, I want to destroy!

The Big Idea (!)


Lexer
(Tokens)
Parser
(AST)
Analyzer
(IR)
Evaluator
(Interpreter)
Generator
(Compiler)

Semantic Analysis

Definition

Semantic analysis checks the meaning of code (per the lang) at *compile-time*.

- Enforces structural AST requirements not handled by the Parser (for IDEs).
  - e.g. Stmt.Assignment receiver.
- Resolves types, variables, functions, and other relevant constructs.
- Performs type checking, which identifies unsafe usages of data.

```
   Compiling error-messages v0.1.0 (/Users/ep/src/rust/error-messages)
error[E0502]: cannot borrow `x` as immutable because it is also borrowed as mutable
 --> src/lib.rs:5:23
  |
3 |     let y = &mut x;
  |             ------ mutable borrow occurs here
4 |
5 |     println!("{} {}", x, y);
  |                       ^  - mutable borrow later used here
  |                       |
  |                       immutable borrow occurs here

error: aborting due to previous error

For more information about this error, try `rustc --explain E0502`.
error: could not compile `error-messages`.

To learn more, run the command again with --verbose.
```

```
//requires static scoping:

LET scope = "definition";

DEF function() DO
  //child of definition
  print("child of " + scope);
END

DEF main() DO
  LET scope = "invocation";
  function();
END

main(); //child of definition
```

```
//performs analysis, not evalution:

LET sum = 0;

FOR i IN range(1, 5) DO
  sum = sum + i; //visited once
END

print(sum);
```

Semantic Analysis Scoping

```
//visit(body) during declaration:

DEF main() DO
  LET scope = "invocation";
  function(); //error: undefined!
END

DEF function() DO
  LET scope = "invocation";
END
```

```
//visit(body) after hoisting:

DECLARE main();
DECLARE function();

DEF main() DO
  LET scope = "invocation";
  function(); //okay!
END

DEF function() DO
  LET scope = "invocation";
END
```

Semantic Analysis Order

2.

Types

I don’t want to create, I want to destroy!

What's in a Type?

Definition

A **type** is a formal description of the meaning of data and valid operations.

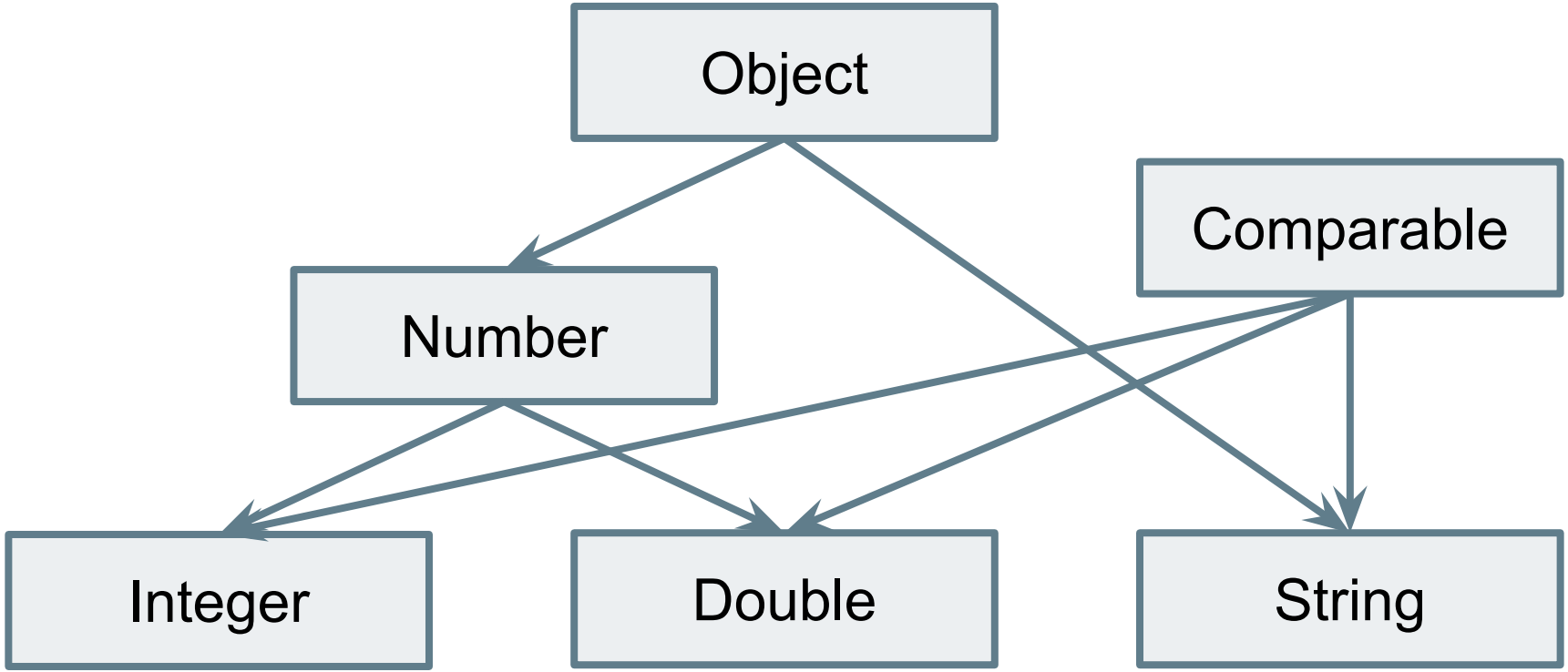
- Types can represent "simple" values, like literals or the type of variables.
- Types may also represent "complex" ideas, like functions or subtyping.

A **type system** defines kinds of types and their associated relations.

"Hello World" : String

42 : Integer x : A true : Boolean

$f(x, y) = x + y : (T, T) \rightarrow T$



A Simple* Type System

Strong Generate an error	**Weak** Coercion or dummy result
Validate on compilation Static	Validate during runtime Dynamic

Types of Type Systems

Rhova
Rust
STRONG
Kotlin
Swift
Elm
Julia
Elixir
Erlang
C#
Go
Scala
Java
Clojure
Groovy
Haskell
Python
Ruby
F#
DYNAMIC
STATIC
Perl
PHP
C
VB
C++
JavaScript
WEAK

Types handle a *subset* of behavior

Some operations are difficult/annoying to type (div 0)

Some must be handled at runtime! (FileNotFound)

Module 5: Analysis

Semantic Analysis

Imagine, like, the Evaluator, but like, without evaluating.

Slides Carnival

Free templates for all your presentation needs

For PowerPoint and Google Slides

100% free for personal or commercial use

Ready to use, professional and customizable

Blow your audience away with attractive visuals